

21-Day Python Programming Challenge

BCA CSEG1021 - Units 3 to 6 Comprehensive Practice

Course-Aligned Challenge Structure

This 21-day challenge is specifically designed to match your BCA Python Programming course syllabus (Units 3-6). Each day's questions directly correspond to topics from your course plan, progressing from foundational to advanced concepts.

Challenge Progression:

- Days 1-7: Unit III - Modules, Packages & Regular Expressions + Basic DSA
- Days 8-14: Unit IV - File Handling & Exception Handling + Intermediate DSA
- Days 15-21: Units V & VI - OOP + NumPy, Pandas, Matplotlib + Advanced DSA

Note: All questions require substantial implementation (20+ lines of code) with proper documentation, error handling, and testing.

Unit III: Modules, Packages and Regular Expressions

Day 1: Introduction to Modules

Difficulty: Easy | **Topics:** Creating modules, import mechanisms, module documentation

Question 1: Mathematical Operations Module

Create a module 'math_operations.py' that includes:

- Functions: factorial(n), fibonacci(n) - returns nth Fibonacci number, is_prime(n), prime_factors(n), gcd(a,b), lcm(a,b)
- Each function should have comprehensive docstrings
- Input validation for all parameters
- Create a test program that imports and demonstrates all functions

Test with: factorial(10), fibonacci(15), is_prime(97), prime_factors(84), gcd(48,18), lcm(12,15)

Question 2: String Utilities Module

Create 'string_utils.py' module with functions:

- count_vowels(text), count_consonants(text) - return count of vowels/consonants
- reverse_words(sentence) - reverse word order in sentence
- is_palindrome(text) - check palindrome ignoring case and spaces
- word_frequency(text) - return dictionary of word counts
- remove_duplicates(text) - remove duplicate words while maintaining order

Write main program demonstrating all functions with sample text inputs.

Question 3: Different Import Methods

Create a module 'conversions.py' with temperature, length, and weight conversion functions. Write three different programs demonstrating: (1) import module_name (2)

from module_name import specific_function (3) from module_name import * and (4) import module_name as alias. Explain the differences and when to use each.

Day 2: Standard Modules - sys and os

Difficulty: Easy | **Topics:** sys module, os module, working with directories

Question 1: System Information Tool (sys module)

Using sys module, create a program that displays:

- Python version (sys.version)
- Platform information (sys.platform)
- Maximum integer size (sys.maxsize)
- Command line arguments (sys.argv) - create a calculator that takes numbers as command line arguments
- Python path (sys.path)

Format the output in a professional report style with clear sections and headers.

Question 2: Directory Management Tool (os module)

Create a directory management program using os module:

- Display current working directory (os.getcwd())
- List all files and directories in current path (os.listdir())
- Create new directory (os.mkdir())
- Change directory (os.chdir())
- Remove directory (os.rmdir())
- Check if path exists (os.path.exists())
- Get file size (os.path.getsize())

Create a menu-driven interface for all operations with proper error handling.

Question 3: File Analysis Tool

Using os and os.path modules, create a tool that: walks through directory tree (os.walk()), counts total files and folders, categorizes files by extension, calculates total directory size, finds largest and smallest files, displays file creation and modification times (os.path.getctime(), os.path.getmtime()). Present results in formatted table.

Day 3: Standard Modules - math and time

Difficulty: Easy | **Topics:** math module, time module, performance measurement

Question 1: Scientific Calculator (math module)

Build a calculator using math module functions:

- Trigonometric: sin, cos, tan, asin, acos, atan (with degree to radian conversion)
- Logarithmic: log, log10, log2
- Power and roots: sqrt, pow, exp
- Rounding: ceil, floor, trunc
- Special: factorial, gcd, fabs, fmod
- Constants: pi, e, tau, inf, nan

Create menu-driven interface with input validation and formatted output.

Question 2: Time and Date Operations (time module)

Create programs using time module:

- Display current time using `time.time()`, `time.ctime()`, `time.localtime()`
- Format time using `time.strftime()` with different format codes
- Create countdown timer using `time.sleep()`
- Calculate age in years, months, days from birth date
- Find day of week for any given date

Display results in user-friendly format with proper labels.

Question 3: Performance Benchmarking Tool

Create a benchmarking tool using `time` module: measure execution time of different algorithms (linear search vs binary search, bubble sort vs quick sort), compare performance with various input sizes (100, 1000, 10000 elements), display results in formatted table showing time in milliseconds, calculate speedup ratios, generate performance report. Use `time.perf_counter()` for accurate timing.

Day 4: Creating Packages

Difficulty: Easy-Medium | **Topics:** Package structure, `__init__.py`, nested packages

Question 1: Geometry Package

Create a package structure:

```
geometry/
  __init__.py
  twod/
    __init__.py
    circle.py (area, circumference, diameter functions)
    rectangle.py (area, perimeter, diagonal functions)
    triangle.py (area, perimeter functions)
  threed/
    __init__.py
    sphere.py (volume, surface_area functions)
    cube.py (volume, surface_area functions)
    cylinder.py (volume, surface_area functions)
```

The `__init__.py` files should expose the key functions. Create test program importing and using all modules.

Question 2: Student Management Package

Create package 'student_mgmt':

- `student.py` - Student class with attributes (name, roll_no, marks, grade)
- `operations.py` - `add_student()`, `delete_student()`, `search_student()`, `update_marks()`
- `reports.py` - `generate_marksheet()`, `calculate_percentage()`, `find_topper()`, `class_average()`
- `storage.py` - `save_to_file()`, `load_from_file()` using JSON
- `__init__.py` - expose main functionality

Create main application demonstrating complete student management workflow.

Question 3: Calculator Package with Sub-packages

Design 'calculator' package with sub-packages: `basic/` (add, subtract, multiply, divide), `scientific/` (trigonometry, logarithm, power), `statistics/` (mean, median, mode, `std_dev`),

conversion/ (temperature, length, weight). Each module should have multiple related functions. Proper `__init__.py` files at each level. Create comprehensive test suite.

Day 5: Regular Expressions - Basics

Difficulty: Easy-Medium | **Topics:** re module, match(), search(), findall()

Question 1: Pattern Matching Basics

Write programs demonstrating:

- `re.match()` - check if string starts with pattern (validate username starting with letter)
- `re.search()` - find pattern anywhere in string (find first phone number in text)
- `re.findall()` - find all occurrences (extract all email addresses from text)
- `re.finditer()` - iterator over all matches (find all dates with positions)
- Demonstrate difference between `match()` and `search()`

Test each function with sample text and explain when to use each.

Question 2: Email Validator

Create email validation program using regex:

- Pattern: username (alphanumeric + underscore/dot), @ symbol, domain name, .extension
- Validate format: `user@domain.com`
- Extract all emails from given text using `findall()`
- Separate emails by domain (.com, .org, .edu, etc.)
- Count valid vs invalid emails

Test with various email formats and edge cases.

Question 3: Phone Number Extractor

Write program to extract phone numbers from text: recognize formats (123-456-7890, (123) 456-7890, 123.456.7890, +91-9876543210), validate Indian mobile numbers (10 digits starting with 6-9), extract all phone numbers using `findall()`, standardize format to common pattern, separate by country code. Display results in organized manner.

Day 6: Regular Expressions - Advanced

Difficulty: Medium | **Topics:** Meta characters, character classes, groups, sub()

Question 1: Meta Characters Practice

Create programs using meta characters:

- `.` (dot) - match any character except newline
- `^` - match start of string, `$` - match end of string
- `*` - 0 or more, `+` - 1 or more, `?` - 0 or 1
- `{m,n}` - between m and n occurrences
- `[]` - character class, `[^]` - negated character class
- `\d`, `\D`, `\w`, `\W`, `\s`, `\S` - character classes

For each meta character, provide 3 practical examples with explanations.

Question 2: Groups and Capturing

Write programs demonstrating groups:

- Extract date components (day, month, year) from DD-MM-YYYY format using groups
- Parse log entries: extract timestamp, level, message separately
- Extract name, email, phone from contact info using named groups
- Access groups using group(), groups(), groupdict()

Test with various input formats and display extracted components.

Question 3: Text Replacement with sub()

Create text processing tool using re.sub(): replace all dates from DD/MM/YYYY to YYYY-MM-DD format, mask credit card numbers (show only last 4 digits), remove all HTML tags from text, standardize phone numbers to common format, replace multiple spaces with single space, censor profanity (replace with asterisks). Display before and after text for each operation.

Day 7: DSA - Arrays and Strings

Difficulty: Medium | **Topics:** Array manipulation algorithms, string processing

Question 1: Array Rotation

Implement array rotation:

- Left rotate array by d positions (in-place)
- Right rotate array by d positions (in-place)
- Implement using reversal algorithm
- Time complexity: $O(n)$, Space complexity: $O(1)$

Test with arrays: [1,2,3,4,5,6,7] rotated by 2,3,5 positions. Display step-by-step process.

Question 2: Maximum Subarray Sum (Kadane's Algorithm)

Implement Kadane's algorithm:

- Find maximum sum of contiguous subarray
- Also return start and end indices of the subarray
- Handle all negative numbers case
- Compare with brute force approach (time complexity comparison)

Test: [-2,1,-3,4,-1,2,1,-5,4], [5,4,-1,7,8], [-2,-3,-1,-4]

Question 3: String Algorithms

Implement: (a) Check if two strings are anagrams (b) Find longest palindromic substring in string (c) Remove duplicate characters from string maintaining order (d) Check if string is rotation of another string (e) Find first non-repeating character. Test each with multiple examples and analyze time complexity.

Unit IV: File and Exception Handling

Day 8: File Handling - Basic Operations

Difficulty: Medium | **Topics:** File access modes, read/write operations

Question 1: File Access Modes

Demonstrate all file access modes:

- 'r' - read mode (default), file must exist

- 'w' - write mode, creates new or overwrites existing
- 'a' - append mode, adds to end of file
- 'r+' - read and write mode
- 'w+' - write and read mode
- 'a+' - append and read mode

For each mode, write example program showing behavior. Demonstrate file pointer position in each mode.

Question 2: Reading Data from Files

Create programs demonstrating different read methods:

- read() - read entire file as single string
- read(n) - read n characters
- readline() - read single line
- readlines() - read all lines into list
- Iterating over file object

Compare memory usage and performance for large files. When to use each method?

Question 3: Writing Data to Files

Create student records program: accept student details (name, roll, marks in 5 subjects), calculate total and percentage, write to file in formatted manner, append new records without overwriting, read and display all records in tabular format. Use write(), writelines() appropriately. Ensure proper file closing.

Day 9: File Handling - Advanced

Difficulty: Medium | **Topics:** with statement, file methods, seek/tell

Question 1: Context Manager - with Statement

Demonstrate with statement:

- Explain advantage of using with statement (automatic file closing)
- Compare traditional file handling vs with statement
- Handle multiple files simultaneously using with
- Copy content from one file to another using with

Create file merge program: combine multiple text files into one using with statement.

Question 2: File Methods and Attributes

Demonstrate file methods and attributes:

- file.seek(offset, whence) - change file position (whence: 0-beginning, 1-current, 2-end)
- file.tell() - return current file position
- file.name - get file name
- file.mode - get file opening mode
- file.closed - check if file is closed

Create program to read specific lines from file using seek() and tell().

Question 3: Working with Directories

Create directory management program: use os module to create directory structure, list all .txt files in directory, rename files based on pattern, copy files from one directory to

another, organize files by extension into separate folders, calculate total size of all files in directory tree. Include error handling for file operations.

Day 10: File Applications

Difficulty: Medium-Hard | **Topics:** CSV files, JSON files, practical applications

Question 1: CSV File Operations

Create employee management using CSV:

- Store: emp_id, name, department, salary, joining_date
- Read CSV file and display in tabular format
- Add new employee record
- Update salary for employee by ID
- Delete employee record
- Search by department and display all employees
- Calculate average salary by department

Use csv module or manual parsing. Create menu-driven interface.

Question 2: JSON File Operations

Create contact manager using JSON:

- Store contacts with: name, phone, email, address
- Load contacts from JSON file
- Add new contact
- Search contact by name/phone
- Update contact details
- Delete contact
- Export contacts to CSV

Use json module. Ensure proper indentation in JSON file.

Question 3: Log File Analyzer

Read server log file containing entries with timestamp, log level (INFO, WARNING, ERROR), and message. Parse log file: count entries by log level, extract all ERROR entries to separate file, find entries within specific time range, search for specific keywords in messages, generate summary report with statistics. Handle large log files efficiently using line-by-line reading.

Day 11: Exception Handling - Basics

Difficulty: Medium | **Topics:** try, except, else, finally blocks

Question 1: Understanding try-except-else-finally

Create programs demonstrating:

- Basic try-except block (handle division by zero)
- try-except-else (else executes if no exception)
- try-except-finally (finally always executes)
- Complete structure: try-except-else-finally

For each, explain execution flow with examples showing when exception occurs and when it doesn't.

Question 2: Handling Multiple Exceptions

Create calculator with exception handling:

- Handle ZeroDivisionError
- Handle ValueError (invalid input)
- Handle OverflowError (result too large)
- Use multiple except blocks
- Use tuple of exceptions in single except
- Generic exception handler (except Exception)

Display appropriate error messages for each exception type.

Question 3: File Operations with Exception Handling

Create file reader with comprehensive error handling: FileNotFoundError (file doesn't exist), PermissionError (no read permission), IOError (read/write errors), UnicodeDecodeError (encoding issues), IsADirectoryError (path is directory). Use finally block to ensure file closure. Provide user-friendly error messages. Test with various error scenarios.

Day 12: Exception Handling - Advanced

Difficulty: Medium-Hard | **Topics:** raise, assert, exception hierarchy

Question 1: Raising Exceptions

Create age validation program:

- Accept age from user
- Raise ValueError if age is negative
- Raise ValueError if age > 150 (unrealistic)
- Raise TypeError if input is not integer
- Create voting eligibility checker (raise exception if age < 18)

Demonstrate raising exceptions with custom messages.

Question 2: Assert Statement

Use assert for validation:

- Create function to calculate square root (assert input >= 0)
- Create list index accessor (assert index within range)
- Create grade calculator (assert marks between 0-100)
- Explain when to use assert vs raise

Show what happens when assertion fails.

Question 3: Custom Exception Classes

Create banking application with custom exceptions: InsufficientFundsError (withdrawal > balance), InvalidAccountError (account not found), NegativeAmountError (negative deposit/withdrawal). Implement BankAccount class with deposit(), withdraw(), transfer() methods. Each method should raise appropriate custom exception. Handle these exceptions in main program with meaningful messages.

Day 13: DSA - Stack Implementation

Difficulty: Medium | **Topics:** Stack data structure and applications

Question 1: Stack Implementation

Implement Stack class with methods:

- push(item) - add item to top
- pop() - remove and return top item
- peek() - return top item without removing
- is_empty() - check if stack is empty
- size() - return number of elements

Implement using list. Include exception handling for empty stack operations.

Question 2: Balanced Parentheses Checker

Using Stack, check if parentheses are balanced:

- Handle (), {}, [] brackets
- Check proper opening and closing order
- Return True if balanced, False otherwise

Test: '[(())]' (True), '[(())]' (False), '{{[[((()))]]}' (True)

Question 3: Infix to Postfix Conversion

Convert infix expression to postfix using stack: handle operators (+, -, *, /), consider operator precedence and associativity, handle parentheses, convert and evaluate postfix expression. Test: 'A+B*C' → 'ABC*+', '(A+B)*C' → 'AB+C*', 'A+B*C-D/E' → 'ABC*+DE/-'

Day 14: DSA - Queue and Recursion

Difficulty: Medium-Hard | **Topics:** Queue data structure, recursion problems

Question 1: Queue Implementation

Implement Queue class:

- enqueue(item) - add to rear
- dequeue() - remove from front
- front() - view front element
- is_empty(), size()
- Also implement Circular Queue with fixed size

Test both implementations with various operations.

Question 2: Recursion Problems

Solve using recursion:

- Tower of Hanoi (move n disks from source to destination)
- Generate all permutations of string
- Binary search (recursive implementation)
- Sum of digits (recursive)

For each, show both recursive and iterative solutions. Compare approaches.

Question 3: Linked List Basics

Implement Singly Linked List: create Node class, implement LinkedList class with insert_at_beginning(), insert_at_end(), delete_by_value(), search(), display() methods. Test with insertion, deletion, searching operations. Count nodes, reverse list, find middle element.

Unit V: Classes and Objects in Python

Day 15: Classes and Objects - Basics

Difficulty: Medium-Hard | **Topics:** Class definition, objects, methods, constructor

Question 1: Student Class

Create Student class:

- Attributes: name, roll_no, marks (list of 5 subjects)
- `__init__()` constructor to initialize
- Methods: `calculate_total()`, `calculate_percentage()`, `calculate_grade()`
- `display()` method to show student details

Create 5 student objects and display their details with grades.

Question 2: Bank Account Class

Create BankAccount class:

- Attributes: `account_number`, `account_holder`, `balance`
- Methods: `deposit(amount)`, `withdraw(amount)`, `check_balance()`
- Validate: withdrawal doesn't exceed balance, amounts are positive
- Display transaction messages

Test with multiple accounts performing various transactions.

Question 3: Class Variables vs Instance Variables

Create Employee class demonstrating: instance variables (`name`, `emp_id`, `salary`) unique to each employee, class variable (`company_name`, `employee_count`) shared by all. Increment `employee_count` in constructor. Methods: `display_details()`, `give_raise(percentage)`. Create multiple employees and show both types of variables.

Day 16: Special Methods and Encapsulation

Difficulty: Hard | **Topics:** `__str__`, `__repr__`, private members, property

Question 1: Special Methods

Create Book class with special methods:

- `__init__(title, author, price, pages)`
- `__str__()` for user-friendly string representation
- `__repr__()` for developer-friendly representation
- `__eq__()` to compare books by ISBN
- `__lt__()` to compare by price

Demonstrate all special methods with examples.

Question 2: Public and Private Data Members

Create Person class:

- Public: `name`, `age`
- Private: `__ssn`, `__salary` (use double underscore)
- Getter and setter methods for private members
- Demonstrate name mangling

Show that private members cannot be accessed directly from outside class.

Question 3: Property Decorators

Create Temperature class: store temperature in Celsius (private), use `@property` decorator to get temperature, `@temperature.setter` to set with validation (not below -

273.15), provide fahrenheit property (calculated from celsius), kelvin property. Demonstrate getter/setter usage like attributes rather than methods.

Day 17: Inheritance

Difficulty: Hard | **Topics:** Single, multiple, multilevel inheritance

Question 1: Single Inheritance

Create inheritance hierarchy:

- Base class: Vehicle (brand, model, year, start(), stop())
- Derived class: Car (num_doors, trunk_size, additional methods)
- Override start() method in Car
- Use super() to call parent class methods

Demonstrate method overriding and super() usage.

Question 2: Multiple Inheritance

Create multiple inheritance:

- Class Father with properties: height, eye_color
- Class Mother with properties: hair_color, skin_tone
- Class Child inherits from both Father and Mother
- Demonstrate Method Resolution Order (MRO)
- Use __mro__ attribute to show resolution order

Handle method name conflicts in multiple inheritance.

Question 3: Multilevel Inheritance

Create multilevel hierarchy: Animal (eat(), sleep()) → Mammal (give_birth()) → Dog (bark(), breed). Each level adds new attributes and methods. Override methods at each level. Demonstrate calling methods from all levels on Dog object. Show isinstance() and subclass() usage.

Day 18: Polymorphism and Abstract Classes

Difficulty: Hard | **Topics:** Method overriding, operator overloading, ABC

Question 1: Method Overriding (Polymorphism)

Demonstrate polymorphism:

- Base class: Shape with area() method
- Derived: Circle, Rectangle, Triangle - each overrides area()
- Create list of different shapes
- Iterate and call area() - polymorphic behavior

Show how same method call produces different results.

Question 2: Operator Overloading

Create Complex number class:

- __add__() for + operator
- __sub__() for - operator
- __mul__() for * operator
- __eq__() for == operator
- __str__() for string representation

Test: c1 = Complex(2,3), c2 = Complex(4,5), c3 = c1 + c2

Question 3: Abstract Base Classes

Use ABC module: create abstract class Payment with abstract methods process_payment(), validate(). Derive CreditCard, DebitCard, UPI classes implementing abstract methods. Show that abstract class cannot be instantiated. Create list of different payment objects and process each polymorphically.

Unit VI: Data Analysis and Visualization

Day 19: NumPy Arrays

Difficulty: Hard | **Topics:** ndarray, array operations, functions

Question 1: Array Creation and Attributes

Create NumPy arrays using:

- np.array(), np.zeros(), np.ones(), np.empty()
- np.arange(), np.linspace()
- np.random.random(), np.random.randint()
- Display attributes: shape, size, ndim, dtype
- Demonstrate reshape(), transpose(), flatten()

Compare performance: NumPy array vs Python list for large data.

Question 2: Array Operations and Broadcasting

Perform operations:

- Arithmetic: add, subtract, multiply, divide arrays
- Universal functions: np.sqrt(), np.exp(), np.log()
- Statistical: np.mean(), np.std(), np.sum(), np.min(), np.max()
- Demonstrate broadcasting with arrays of different shapes

Matrix multiplication using np.dot()

Question 3: Array Indexing and Slicing

Create 2D array: demonstrate indexing (arr[row, col]), slicing (arr[start:end, start:end]), boolean indexing (arr[arr > 5]), fancy indexing (arr[[0,2,4]]). Perform operations: extract row, extract column, extract submatrix, modify elements using conditions. Sort using np.sort(), search using np.where().

Day 20: Pandas DataFrames

Difficulty: Hard | **Topics:** Series, DataFrame, operations

Question 1: Series and DataFrame Basics

Work with Pandas:

- Create Series from list, dictionary, array
- Create DataFrame from dictionary, list of lists, CSV file
- Operations: head(), tail(), info(), describe()
- Select columns, add new column, delete column

Demonstrate indexing: loc[], iloc[], at[], iat[]

Question 2: DataFrame Operations

Perform operations:

- Binary operations: `add()`, `sub()`, `mul()`, `div()`
- Handle missing data: `isnull()`, `dropna()`, `fillna()`
- Aggregation: `groupby()`, `sum()`, `mean()`, `count()`
- Sorting: `sort_values()`, `sort_index()`
- Filtering: boolean indexing, `query()`

Merge/join DataFrames using `merge()`, `concat()`

Question 3: Data Import/Export

Create student data analysis: read CSV with student details, clean data (handle missing values, remove duplicates), add calculated columns (total, percentage, grade), group by class/section, calculate statistics (average, topper, pass percentage), export results to new CSV and Excel. Display formatted reports.

Day 21: Matplotlib Visualization

Difficulty: Hard | **Topics:** Different plot types, customization, subplots

Question 1: Basic Plots

Create various plots:

- Line plot: plot student marks across exams
- Bar chart: compare marks of different students
- Histogram: show mark distribution
- Scatter plot: correlation between two subjects
- Pie chart: grade distribution

Add: title, labels, legend, grid, colors, markers

Question 2: Advanced Customization

Customize plots:

- Multiple lines in same plot with different styles
- Set axis ranges, scales (linear/log)
- Add annotations and text
- Customize colors, line styles, markers
- Save figure in different formats (PNG, PDF, SVG)

Create publication-quality plots

Question 3: Subplots Dashboard

Create comprehensive dashboard using subplots: 2x2 grid with (1) line plot showing trend over time, (2) bar chart comparing categories, (3) histogram showing distribution, (4) pie chart showing proportions. Use `plt.subplot()` or `plt.subplots()`. Share axes where appropriate. Add overall title with `suptitle()`. Adjust spacing with `tight_layout()`.

Tips for Success

1. Follow the syllabus topics in order
2. Write code with proper documentation and comments
3. Test every program with multiple test cases
4. Include error handling in all file/input operations
5. Practice DSA problems daily for better problem-solving
6. Compare your solutions with standard implementations

7. Focus on understanding concepts, not memorizing code
8. Build on previous days' knowledge progressively

Complete all 21 days for comprehensive course mastery!

Best wishes for your BCA Python Programming journey!